JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class MutableCallSite

java.lang.Object
    java.lang.invoke.CallSite
        java.lang.invoke.MutableCallSite

Direct Known Subclasses:

AbstractRelinkableCallSite

```
public non-sealed class MutableCallSite
extends CallSite
```

A `MutableCallSite` is a `CallSite` whose target variable behaves like an ordinary field. An `invokedynamic` instruction linked to a `MutableCallSite` delegates all calls to the site's current target. The dynamic invoker of a mutable call site also delegates each call to the site's current target.

Here is an example of a mutable call site which introduces a state variable into a method handle chain.

```
MutableCallSite name = new MutableCallSite(MethodType.methodType(String.class
MethodHandle MH_name = name.dynamicInvoker();
MethodType MT_str1 = MethodType.methodType(String.class);
MethodHandle MH_upcase = MethodHandles.lookup()
    .findVirtual(String.class, "toUpperCase", MT_str1);
MethodHandle worker1 = MethodHandles.filterReturnValue(MH_name, MH_upcase);
name.setTarget(MethodHandles.constant(String.class, "Rocky"));
assertEquals("ROCKY", (String) worker1.invokeExact());
name.setTarget(MethodHandles.constant(String.class, "Fred"));
assertEquals("FRED", (String) worker1.invokeExact());
// (mutation can be continued indefinitely)
```

The same call site may be used in several places at once.

```
MethodType MT_str2 = MethodType.methodType(String.class, String.class);
MethodHandle MH_cat = lookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
MethodHandle MH_dear = MethodHandles.insertArguments(MH_cat, 1, ", dear?");
MethodHandle worker2 = MethodHandles.filterReturnValue(MH_name, MH_dear);
assertEquals("Fred, dear?", (String) worker2.invokeExact());
name.setTarget(MethodHandles.constant(String.class, "Wilma"));
assertEquals("WILMA", (String) worker1.invokeExact());
```

```
assertEquals("Wilma, dear?", (String) worker2.invokeExact());
```

Non-synchronization of target values: A write to a mutable call site's target does not force other threads to become aware of the updated value. Threads which do not perform suitable synchronization actions relative to the updated call site may cache the old target value and delay their use of the new target value indefinitely. (This is a normal consequence of the Java Memory Model as applied to object fields.)

The `syncAll` operation provides a way to force threads to accept a new target value, even if there is no other synchronization.

For target values which will be frequently updated, consider using a volatile call site instead.

Since:

1.7

Constructor Summary

Constructors

Constructor	Description
MutableCallSite(MethodHandle target)	Creates a call site object with an initial target method handle.
MutableCallSite(MethodType type)	Creates a blank call site object with the given method type.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
final MethodHandle	**dynamicInvoker**()	Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.
final MethodHandle	**getTarget**()	Returns the target method of the call site, which behaves like a

		normal field of the `MutableCallSite`.
`void`	**`setTarget`**`(MethodHandle newTarget)`	Updates the target method of this call site, as a normal variable.
`static void`	**`syncAll`**`(MutableCallSite[] sites`	Performs a synchronization operation on each call site in the given array, forcing all other threads to throw away any cached values previously loaded from the target of any of the call sites.

Methods declared in class CallSite

`type`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

MutableCallSite

`public MutableCallSite(MethodType type)`

Creates a blank call site object with the given method type. The initial target is set to a method handle of the given type which will throw an `IllegalStateException` if called.

The type of the call site is permanently set to the given type.

Before this `CallSite` object is returned from a bootstrap method, or invoked in some other manner, it is usually provided with a more useful target method, via a call to `setTarget`.

Parameters:

type - the method type that this call site will have

Throws:

NullPointerException - if the proposed type is null

MutableCallSite

```
public MutableCallSite(MethodHandle target)
```

Creates a call site object with an initial target method handle. The type of the call site is permanently set to the initial target's type.

Parameters:

target - the method handle that will be the initial target of the call site

Throws:

NullPointerException - if the proposed target is null

Method Details

getTarget

```
public final MethodHandle getTarget()
```

Returns the target method of the call site, which behaves like a normal field of the MutableCallSite.

The interactions of getTarget with memory are the same as of a read from an ordinary variable, such as an array element or a non-volatile, non-final field.

In particular, the current thread may choose to reuse the result of a previous read of the target from memory, and may fail to see a recent update to the target by another thread.

Specified by:

getTarget in class CallSite

Returns:

the linkage state of this call site, a method handle which can change over time

See Also:

setTarget(MethodHandle)

setTarget

```
public void setTarget(MethodHandle newTarget)
```

Updates the target method of this call site, as a normal variable. The type of the new target must agree with the type of the old target.

The interactions with memory are the same as of a write to an ordinary variable, such as an array element or a non-volatile, non-final field.

In particular, unrelated threads may fail to see the updated target until they perform a read from memory. Stronger guarantees can be created by putting appropriate operations into the bootstrap method and/or the target methods used at any given call site.

Specified by:

`setTarget` in class `CallSite`

Parameters:

`newTarget` - the new target

Throws:

`NullPointerException` - if the proposed new target is null

`WrongMethodTypeException` - if the proposed new target has a method type that differs from the previous target

See Also:

`getTarget()`

dynamicInvoker

```
public final MethodHandle dynamicInvoker()
```

Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.

This method is equivalent to the following code:

```
MethodHandle getTarget, invoker, result;
getTarget = MethodHandles.publicLookup().bind(this, "getTarget", MethodType.
invoker = MethodHandles.exactInvoker(this.type());
result = MethodHandles.foldArguments(invoker, getTarget)
```

Specified by:

`dynamicInvoker` in class `CallSite`

Returns:

a method handle which always invokes this call site's current target

syncAll

```
public static void syncAll(MutableCallSite[] sites)
```

Performs a synchronization operation on each call site in the given array, forcing all other threads to throw away any cached values previously loaded from the target of any of the call sites.

This operation does not reverse any calls that have already started on an old target value. (Java supports forward time travel only.)

The overall effect is to force all future readers of each call site's target to accept the most recently stored value. ("Most recently" is reckoned relative to the `syncAll` itself.) Conversely, the `syncAll` call may block until all readers have (somehow) decached all previous versions of each call site's target.

To avoid race conditions, calls to `setTarget` and `syncAll` should generally be performed under some sort of mutual exclusion. Note that reader threads may observe an updated target as early as the `setTarget` call that install the value (and before the `syncAll` that confirms the value). On the other hand, reader threads may observe previous versions of the target until the `syncAll` call returns (and after the `setTarget` that attempts to convey the updated version).

This operation is likely to be expensive and should be used sparingly. If possible, it should be buffered for batch processing on sets of call sites.

If `sites` contains a null element, a `NullPointerException` will be raised. In this case, some non-null elements in the array may be processed before the method returns abnormally. Which elements these are (if any) is implementation-dependent.

Java Memory Model details

In terms of the Java Memory Model, this operation performs a synchronization action which is comparable in effect to the writing of a volatile variable by the current thread, and an eventual volatile read by every other thread that may access one of the affected call sites.

The following effects are apparent, for each individual call site `S`:

- A new volatile variable `V` is created, and written by the current thread. As defined by the JMM, this write is a global synchronization event.
- As is normal with thread-local ordering of write events, every action already performed by the current thread is taken to happen before the volatile write to `V`. (In some implementations, this means that the current thread performs a global release operation.)
- Specifically, the write to the current target of `S` is taken to happen before the volatile write to `V`.
- The volatile write to `V` is placed (in an implementation specific manner) in the global synchronization order.

- Consider an arbitrary thread `T` (other than the current thread). If `T` executes a synchronization action `A` after the volatile write to `V` (in the global synchronization order), it is therefore required to see either the current target of `S`, or a later write to that target, if it executes a read on the target of `S`. (This constraint is called "synchronization-order consistency".)
- The JMM specifically allows optimizing compilers to elide reads or writes of variables that are known to be useless. Such elided reads and writes have no effect on the happens-before relation. Regardless of this fact, the volatile `V` will not be elided, even though its written value is indeterminate and its read value is not used.

Because of the last point, the implementation behaves as if a volatile read of `V` were performed by `T` immediately after its action `A`. In the local ordering of actions in `T`, this read happens before any future read of the target of `S`. It is as if the implementation arbitrarily picked a read of `S`'s target by `T`, and forced a read of `V` to precede it, thereby ensuring communication of the new target value.

As long as the constraints of the Java Memory Model are obeyed, implementations may delay the completion of a `syncAll` operation while other threads (`T` above) continue to use previous values of `S`'s target. However, implementations are (as always) encouraged to avoid livelock, and to eventually require all threads to take account of the updated target.

Discussion: For performance reasons, `syncAll` is not a virtual method on a single call site, but rather applies to a set of call sites. Some implementations may incur a large fixed overhead cost for processing one or more synchronization operations, but a small incremental cost for each additional call site. In any case, this operation is likely to be costly, since other threads may have to be somehow interrupted in order to make them notice the updated target value. However, it may be observed that a single call to synchronize several sites has the same formal effect as many calls, each on just one of the sites.

Implementation Note: Simple implementations of `MutableCallSite` may use a volatile variable for the target of a mutable call site. In such an implementation, the `syncAll` method can be a no-op, and yet it will conform to the JMM behavior documented above.

Parameters:

`sites` - an array of call sites to be synchronized

Throws:

`NullPointerException` - if the `sites` array reference is null or the array contains a null

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.